

Entornos de desarrollo

UNIDAD 4

4. Optimización y documentación

Optimización y documentación

Objetivos

- Comprender el concepto de refactorización y los patrones de refactorización más usuales.
- Aplicar patrones de refactorización en el ámbito del entorno de desarrollo.
- Revisar el código fuente mediante un analizador de código.
- Conocer el concepto de control de versiones.
- Utilizar herramientas de control de versiones integradas en el entorno de desarrollo.
- Comprender la necesidad de una correcta documentación de una aplicación.
- Emplear herramientas del entorno de desarrollo para documentar clases.

Contenidos

- 4.1. Refactorización
- 4.2. Analizadores de código
- 4.3. Control de versiones
- 4.4. Documentación

Introducción

La calidad del código creado, como producto final de un proyecto de desarrollo, es fundamental para facilitar la tarea de mantenimiento, la cual supone un porcentaje muy elevado del esfuerzo realizado para el desarrollo de software. Por eso, actualmente, la refactorización del código fuente o mejora de su estructura interna, sin alterar su comportamiento externo, es una tarea muy relevante. En esta unidad, se explicará con detalle dicho proceso y se estudiarán los patrones de refactorización más usuales y su aplicación en Eclipse.

Por otro lado, los analizadores de código sirven para la mejora de la calidad del código fuente, pues detectan defectos en este, que podrían tener efectos adversos. Aquí, se describe el funcionamiento de un analizador de código integrado en el IDE Eclipse.

Un correcto control sobre las diferentes versiones que se van generando de una aplicación informática también es de gran importancia para controlar los cambios que se van produciendo y para facilitar el mantenimiento del software, motivo por el cual se dedica una parte de esta unidad al control de versiones.

La realización de una documentación clara y explicativa de una aplicación informática, incluyendo el código fuente, también es esencial para facilitar la tarea de mantenimiento. En este tema, se contempla la posibilidad de usar las herramientas de documentación que vienen incorporadas en los entornos de desarrollo.

■ 4.1. Refactorización

Uno de los productos finales del proceso de desarrollo de software es el código fuente y, ante cualquier cambio que sea necesario llevar a cabo en una aplicación, habrá que modificar dicho código. Es muy frecuente que, a lo largo del ciclo de vida de una aplicación, haya que realizar cambios sobre esta por varios motivos y, para cada uno de ellos, existen distintos tipos de mantenimiento:

- **Mantenimiento perfectivo:** es el que se lleva a cabo porque el cliente propone nuevos requisitos funcionales o de cualquier otro tipo.
- **Mantenimiento correctivo:** es el que se realiza como consecuencia de la detección por parte del cliente de algún error tras la entrega de la aplicación.
- **Mantenimiento adaptativo:** es el que se lleva a cabo para poder utilizar la aplicación en nuevos entornos de hardware o software, como por ejemplo, el que se requiere cuando se necesita usar un programa en un nuevo sistema operativo.

La REFACTORIZACIÓN

es una práctica esencial en el desarrollo de software que consiste en MEJORAR LA ESTRUCTURA INTERNA DEL CÓDIGO SIN ALTERAR SU COMPORTAMIENTO externo ni su funcionalidad. Su objetivo principal es hacer que el código sea más legible, mantenible y eficiente, reduciendo la deuda técnica acumulada por malas prácticas o atajos en el desarrollo.

Argot técnico



El vocablo **refactorizar** no aparece en el diccionario de la RAE. Proviene del inglés *refactoring*, pero esta palabra inglesa tampoco aparece de momento en los diccionarios de inglés.

Para evitar usar una palabra extranjera, se podrían emplear vocablos como **«perfeccionar»**, **«reestructurar»**, **«reorganizar»**, pero estos términos resultan más genéricos de lo deseable, por lo que a lo largo de esta unidad se usará el término *refactorizar*, cuyo significado se ha señalado en el párrafo anterior.

Argot técnico



El término **bad smell**, que traducido literalmente del inglés quiere decir **‘mal olor’**, se debe entender en sentido metafórico. Es frecuente escuchar la expresión española de que «algo no huele bien», la cual se emplea no en el sentido estricto del verbo oler (‘percibir un olor’), sino en sentido figurado. De hecho, en el diccionario de la RAE, la expresión *no oler bien* significa **«dar sospecha de que encubre un daño o fraude»**. Esto es precisamente lo que ocurre cuando se detecta un *mal olor* en el código: se trata de un síntoma o una sospecha de que hay algo mal en el código que puede tener efectos secundarios negativos.

4.2. Analizadores de código

*Son los avisos
automáticos en
NetBeans*

Los analizadores de código son herramientas que realizan un análisis estático del código fuente. Estos analizadores evalúan el código fuente sin llegar a ejecutarlo. El objetivo es una mejora del código fuente sin modificar su comportamiento, que es exactamente el mismo fin que el de la refactorización. Sin embargo, en el caso de la refactorización, es el programador o la programadora quien debe detectar los síntomas de código mejorable (*bad smells*) y aplicar, entonces, el patrón de refactorización que permite eliminar esos síntomas. Los analizadores de código automáticamente detectan los síntomas y aportan también de forma automática una solución que la persona encargada de la programación puede decidir si aplicar o no.

Los analizadores de código realizan un análisis léxico y sintáctico del código fuente y si detectan que este es mejorable, lo indicarán y propondrán la manera de realizar la mejora.

La función principal de los analizadores de código es encontrar porciones de código que puedan generar efectos adversos como:

- Reducir el rendimiento.
- Provocar errores.
- Crear problemas de seguridad.

4.3. Control de versiones

El software está sometido a continuos cambios, no solo durante las tareas de desarrollo que se llevan a cabo antes de que el producto sea entregado al cliente (análisis, diseño, programación y pruebas), sino también después. Esto ha llevado a que se incluya la tarea de mantenimiento dentro de las fases del desarrollo de una aplicación informática.

Aparte de dicha tarea, ha surgido también la necesidad de otra nueva labor, más de gestión que técnica, que se debe llevar a cabo de manera paralela a las tareas técnicas de desarrollo (análisis, diseño, etc.) y que se conoce **como gestión de la configuración del software (GCS)**. Esta tarea tiene fundamentalmente cuatro objetivos ante cualquier cambio:

1. Identificar el cambio.
2. Controlar el cambio.
3. Garantizar que el cambio se implemente de manera adecuada.
4. Informar de los cambios aplicados a todos aquellos a los que pueda afectar.

*El CONTROL DE VERSIONES se refiere al proceso de
GUARDAR DIFERENTES ARCHIVOS O «VERSIONES» A LO LARGO DE LAS DIFERENTES
ETAPAS DE UN PROYECTO*

Esto permite a los desarrolladores hacer un seguimiento de lo que se ha hecho y volver a una fase anterior si deciden que quieren revertir algunos de los cambios que han hecho.

Esto es útil por varias razones. Por ejemplo, facilita la resolución de errores y la corrección de otros errores que puedan ocurrir durante el desarrollo. También puede anotar los cambios en cada versión, para ayudar a cualquier miembro del equipo a mantenerse al día sobre lo que se ha completado y lo que aún queda por hacer.

GIT es un

SOFTWARE DE CONTROL DE VERSIONES (VCS)

diseñado por Linus Torvalds, pensando en la eficiencia, la confiabilidad y compatibilidad del mantenimiento de versiones de aplicaciones informáticas cuando estas tienen un gran número de archivos de código fuente. Su propósito es llevar registro de los cambios en archivos de computadora incluyendo coordinar el trabajo que varias personas realizan sobre archivos compartidos en un repositorio de código.

*Crear un **REPOSITORIO GIT** en **NETBEANS** es sumamente fácil:*

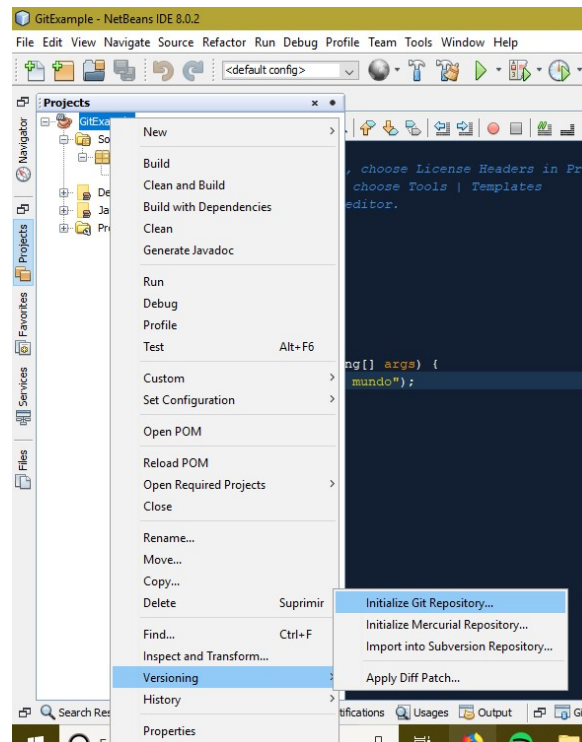
*** GIT debe
estar activado
en NetBeans**

Paso 1: Abre tu proyecto en NetBeans.

Paso 2: Haz clic derecho sobre el nombre de tu proyecto.

*Paso 3: Selecciona **Versioning > Initialize Git Repository**
(**Versiones > Inicializar repositorio Git**).*

*Paso 4: Aparecerá una ventana confirmando la ruta. Haz clic en **OK**.*



Añadir una versión a un *REPOSITORIO GIT* en *NETBEANS* es igualmente fácil:

Paso 1: En la ventana de Proyectos, haz clic derecho sobre el proyecto que acabas de modificar.

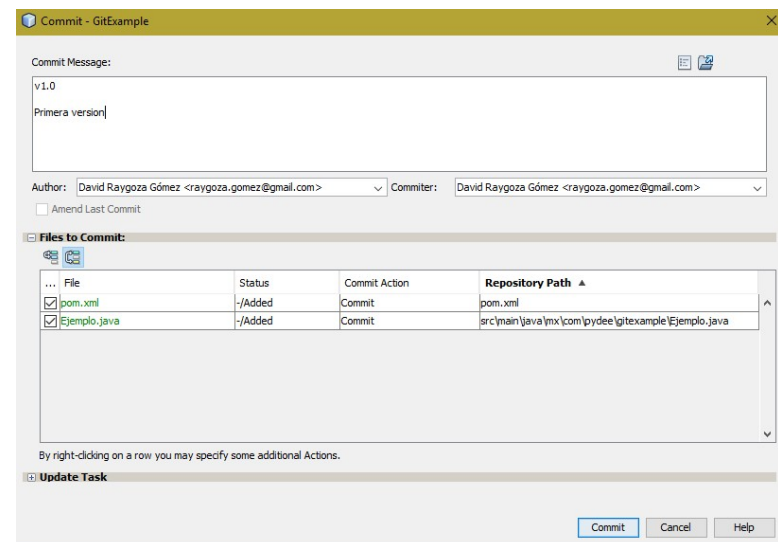
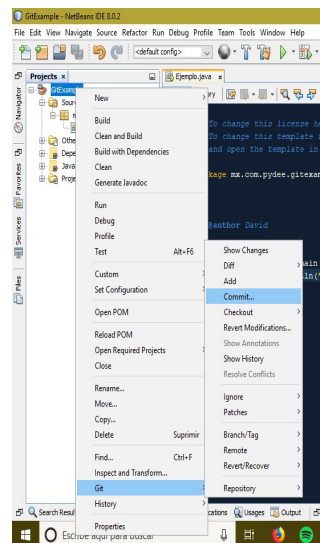
Paso 2: Selecciona *Git > Commit...* (o *Confirmar...*).

Paso 3: Se abrirá una ventana. En la parte superior para escribir un mensaje descriptivo de los cambios realizados.

Paso 4: Asegúrate de marcar las casillas de los archivos que deseas incluir en la nueva versión, lo mejor marcar la casilla superior para seleccionar todos.

Paso 5: Haz clic en el botón *Commit* (o *Confirmar*).

*** GIT debe estar activado en NetBeans**



Acceder a una versión a un **REPOSITORIO GIT en NETBEANS** es igualmente fácil:

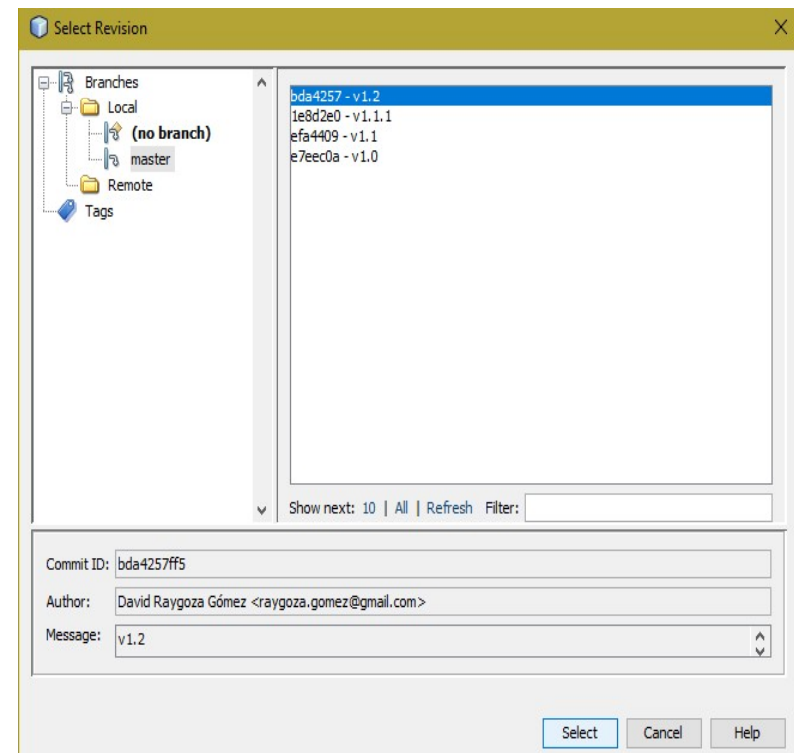
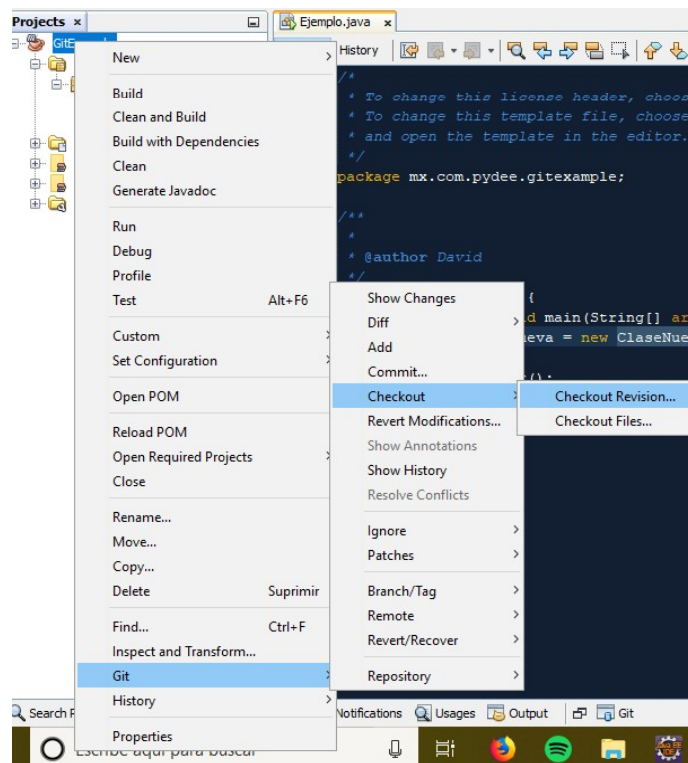
Paso 1: En la ventana de Proyectos, haz clic derecho sobre el proyecto que acabas de modificar.

Paso 2: Selecciona Git > Check-out > Check-out Revision.

Paso 3: Se abrirá una ventana con las diferentes versiones.

Paso 4: Se elige la versión y se pica en el boton Select.

*** GIT debe estar activado en NeatBeans**



GITHUB es una

PLATAFORMA BASADA EN LA NUBE DISEÑADA PARA ALMACENAR, COMPARTIR Y COLABORAR EN PROYECTOS DE DESARROLLO DE SOFTWARE.

Está construida sobre Git, un sistema de control de versiones (VCS) que permite realizar un seguimiento de los cambios en los archivos y facilita el trabajo colaborativo. Al utilizar GitHub podrás:

- *Almacenar tu código en repositorios, lo que permite gestionar y rastrear cambios a lo largo del tiempo.*
- *Colaborar con otros desarrolladores, revisando y sugiriendo mejoras en el código.*
- *Evitar conflictos al trabajar en equipo, ya que Git permite que cada colaborador trabaje en su propia rama sin afectar el trabajo de los demás.*
- *Publicar proyectos para compartirlos con la comunidad o mantenerlos privados para uso interno.*

Crear una cuenta GitHub:

1. Ve a <https://github.com> Y regístrate con tu correo.



Crear una cuenta GitHub:

2. Introduce tu contraseña y tu nombre de usuario.

Email*

jalbfra731@g.educaand.es

Password*

.....

I..Y..9.#

Password should be at least 15 characters OR at least 8 characters number and a lowercase letter.

Username*

JoseM-SS

Username may only contain alphanumeric characters or single hyp cannot begin or end with a hyphen.

Your Country/Region*

Spain

For compliance reasons, we're required to collect country information for you occasional updates and announcements.

Email preferences

Crear una cuenta GitHub:

3. Verifica tu cuenta.

Verify your account

Verify your account

Please solve a puzzle so we can safely create your account.

Visual puzzle

Crear una cuenta GitHub:

4. Se envía un código a tu cuenta que debes proporcionar.

Confirm your email address

We have sent a code to jalbfra731@g.educaand.es

Enter code

1

9

1

3

3

2

5

Continue

Crear una cuenta GitHub:

Y ya estarás registrado y listo para acceder.



Sign in to GitHub

Your account was created successfully! Please sign in to continue.

Username or email address

jalbfra731@g.educaand.es

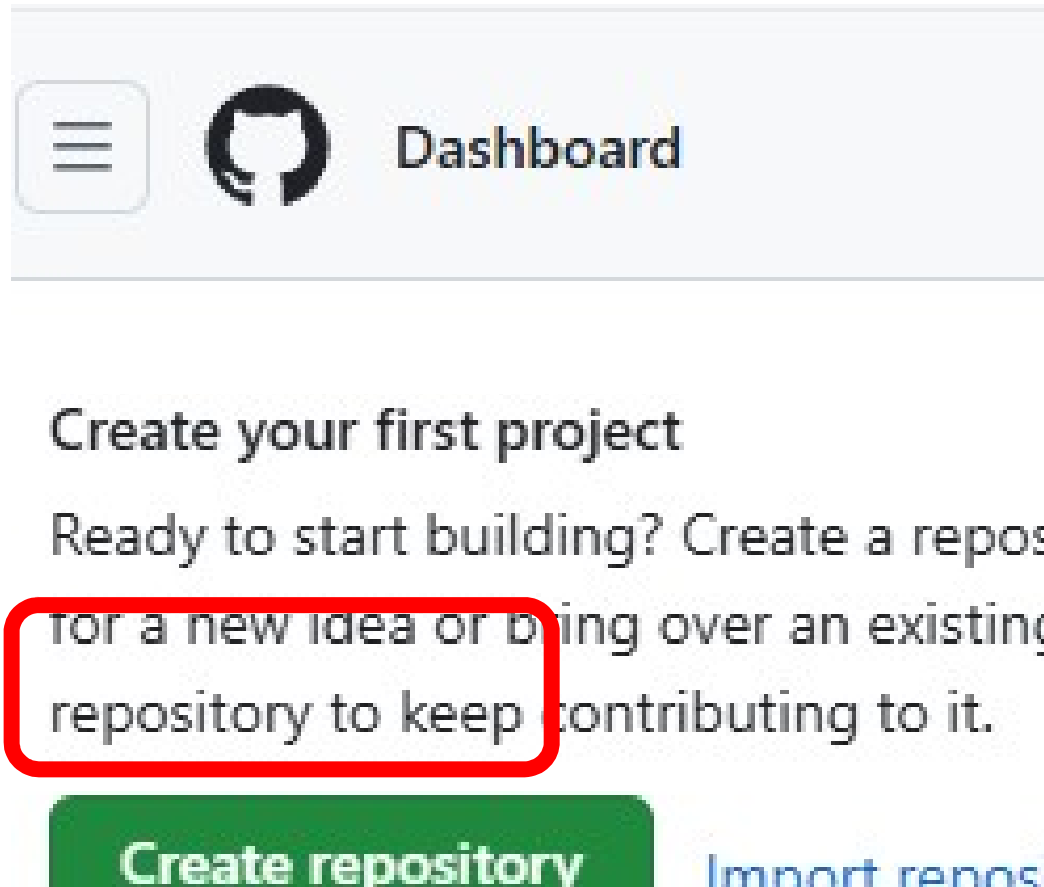
Password

[Forgot password](#)

.....

Crear un repositorio GitHub:

1. Entra en tu cuenta GitHub y pica en “Crear repositorio”.



Crear un repositorio GitHub:

2. Ponle un nombre, indica que será privado y pica en “Crear repositorio”.

1

General

Owner *

JoseM-S

Repository name *

1DAW2526

✓ 1DAW2526 is available.

Great repository names are short and memorable. How about [legendary-umbrella?](#)

Description

Repositorio de 1DAW para el curso 25/26

39 / 350 characters

2

Configuration

Choose visibility *

Choose who can see and commit to this repository

Add README

READMEs can be used as longer descriptions. [About READMEs](#)

Add .gitignore

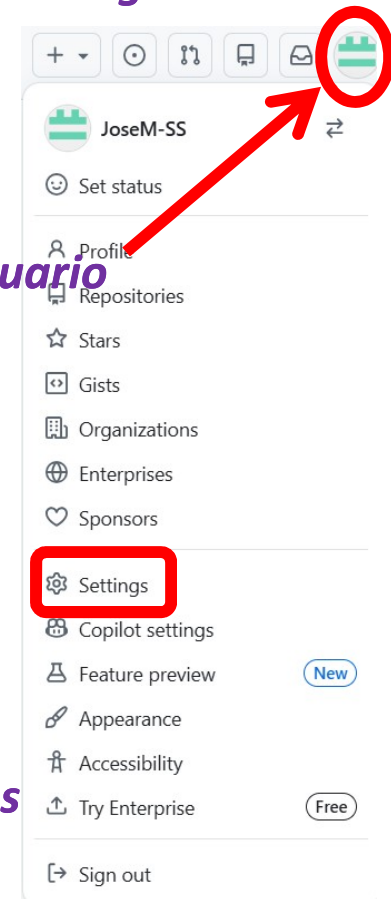
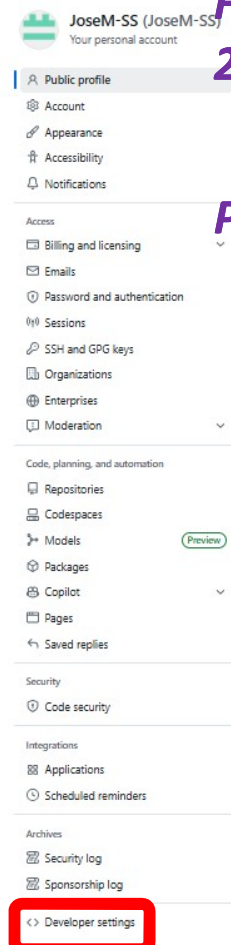
.gitignore tells git which files not to track. [About ignoring files](#)

Generar un TOKEN en GitHub:

Un **TOKEN** en GitHub, comúnmente llamado Personal Access Token (**PAT**), es una cadena alfanumérica segura, alternativa a tu contraseña al usar la API de GitHub, la línea de comandos (Git Bash, terminal) o herramientas de terceros, con mayor seguridad al permitir accesos limitados y revocables. Forma parte de la **POLÍTICA DE SEGURIDAD de GitHub** desde agosto de 2021 para mejorar la protección de datos.

Para crear un PAT habremos de seguir los siguientes pasos:

1. Se inicia la sesión en GitHub entra en el menú del usuario
2. Settings y en el menú del lateral izquierdo:
3. Developer settings > Personal access tokens
4. Tokens (classic) > Generate new token
5. Se asigna un nombre, se indica que no expire en el tiempo y se activan todos los permisos
6. Generate token
7. Se copia de inmediato porque ya no se mostrará más



Personal access tokens (classic)

Generate new token ▼

Tokens you have generated that can be used to access the [GitHub API](#).



Make sure to copy your personal access token now. You won't be able to see it again!

✓ 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 

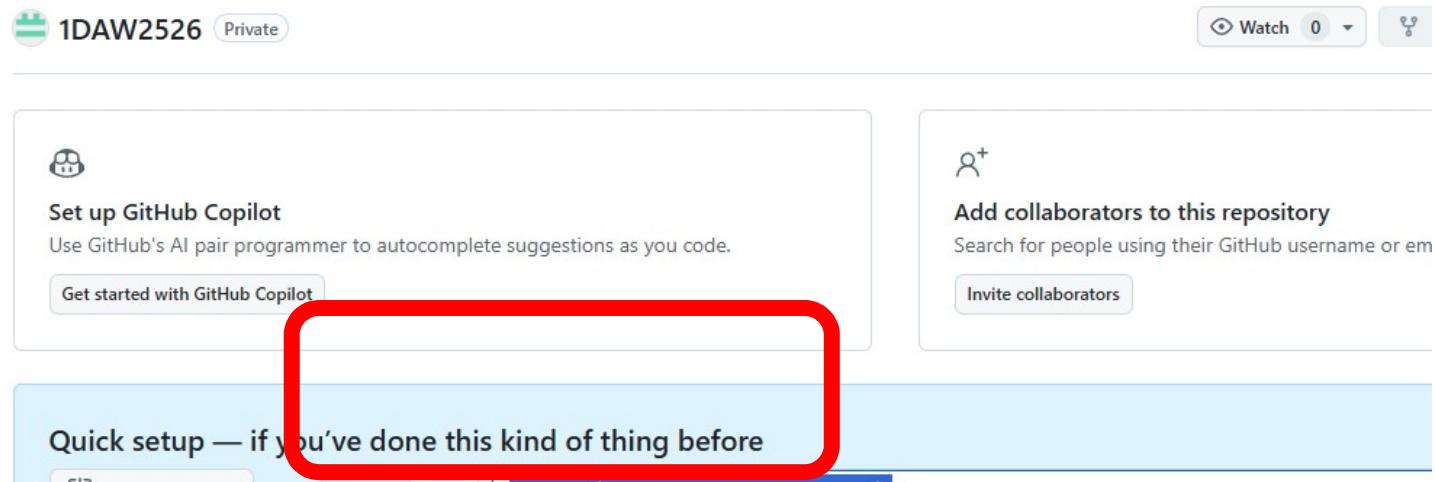
Delete

Personal access tokens (classic) function like ordinary OAuth access tokens. They can be used instead of a password for Git over HTTPS, or can be used to [authenticate to the API over Basic Authentication](#).

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

Integrar un repositorio GitHub en NetBeans:

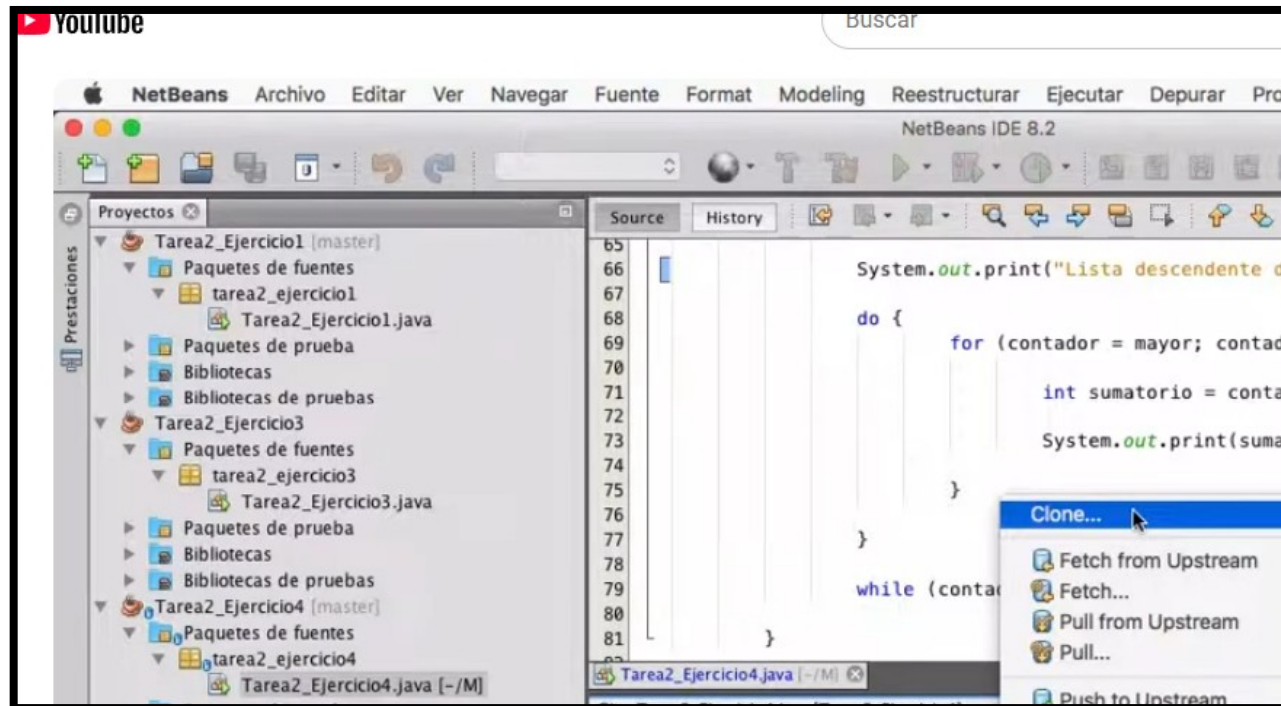
1. Se copia la dirección del repositorio GitHub.



<https://github.com/JoseM-SS/1DAW2526.git>

Integrar un repositorio GitHub en NetBeans:

2. Se accede a clonar el repositorio remoto: Team > Remote > Clone.



Integrar un repositorio GitHub en NetBeans:

3. Se pega la dirección del repositorio GitHub y Siguiente.

Pasos

1. Remote Repository
2. Remote Branches
3. Destination Directory

Remote Repository

Specify Git Repository Location:

Repository URL: https://github.com/monsalvepilas/Tarea1_Ejerc

User: (leave blank for anonymous access)

Password: ☒ Save Password

[Proxy Configuration...](#)

Specify Destination Folder:

Clone into: (Leave empty to specify the destination later)

Integrar un repositorio GitHub en NetBeans:

4. Se introduce el directorio en donde se almacenará y Terminar.

Pasos

1. Remote Repository
2. Remote Branches
3. **Destination Directory**

Destination Directory

Specify the Parent Directory and Name for this Clone

Parent Directory:

Clone Name:

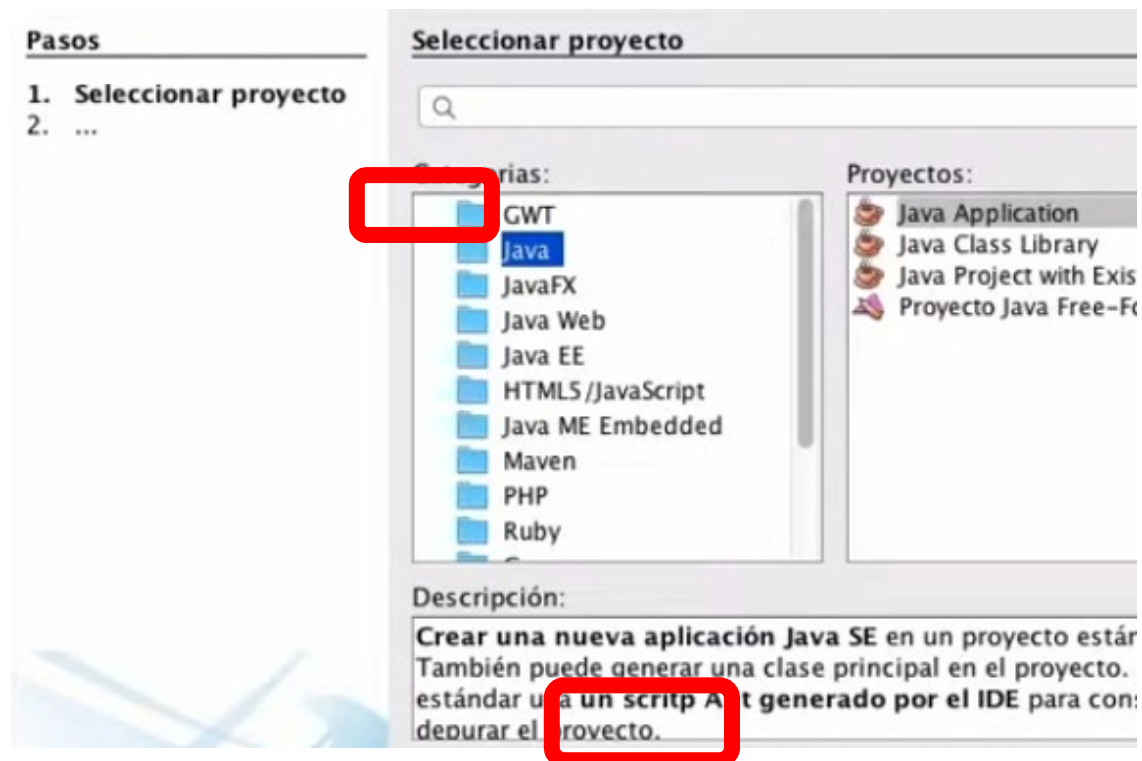
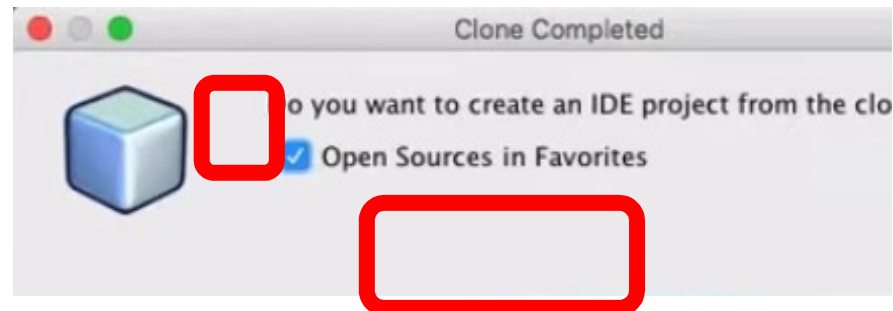
Checkout Branch:

Remote Name:

☒ Scan for NetBeans Projects after Clone

Integrar un repositorio GitHub en NetBeans:

5. Indicamos que se cree el proyecto:



Integrar un repositorio GitHub en NetBeans:

6. Nombre el proyecto y Terminar:

Pasos	Nombre y ubicación
1. Seleccionar proyecto	
2. Nombre y ubicación	

Nombre proyecto:	JavaApplication43
Ubicación del proyecto:	/Users/monsalve/Desktop/Vacaciones/Tarea2_Ejerc
Carpeta proyecto:	Desktop/Vacaciones/Tarea2_Ejercicio5/JavaApplica
☒ Usar una carpeta dedicada para almacenar las bibliotecas	
Carpeta de Bibliotecas:	./lib
Usuarios y proyectos diferentes pueden compartir las mismas librerías de compilación (ver la Ayuda para detalles).	
☒ Crear clase principal	javaapplication43.JavaApplication43

Subir la versión al repositorio GitHub:

1. *Clic derecho sobre el proyecto.*
2. *Selecciona Git> Remote > **PUSH**.*
3. *Se introduce la URL del repositorio GitHub si no se autorellena.*
4. *Se proporciona el token.*
5. *Siguiente > Terminar.*

Bajar la versión al repositorio GitHub:

1. *Clic derecho sobre el proyecto.*
2. *Selecciona Git> Remote > **PULL**.*
3. *Se introduce la URL del repositorio GitHub si no se autorellena.*
4. *Se proporciona el token.*
5. *Siguiente > Terminar.*